	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.C. René Adrián Dávila Pérez

Asignatura: Programación orientada a objetos

Grupo: 07

No de práctica(s): REPORTE DE PROYECTO ETAPA 2

Integrante(s): 424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada: 1

Semestre: 2027 - 1

Fecha de entrega: 26 de octubre de 2026

Observaciones:

Calificación:

Índice general

1 INTRODUCCIÓN 2

1.1 Planteamiento del Problema . . . 2

1.2 Motivación . . . 2

1.3 Objetivos . . . 3

1.3.1 Objetivo General . . . 3

1.3.2 Objetivos Específicos . . . 3

2 MARCO TEÓRICO 4

2.1 Encapsulamiento y Protección de Estado . . . 4

2.2 Modularidad y Espacios de Nombres (Packages) . . . 5

2.3 Jerarquías de Clases y Abstracción . . . 6

2.4 Polimorfismo Dinámico y Late Binding . . . 7

3 DESARROLLO Y ARQUITECTURA DEL SISTEMA 8

3.1 Arquitectura de Paquetes y Distribución Modular . . . 8

3.2 Descripción Algorítmica de la Jerarquía y Clases . . . 9

3.2.1 Clases Abstractas Base (Paciente, Medico, Enfermero) . . . 9

3.2.2 Subclases Especializadas . . . 10

3.2.3 Interfaz AtencionMedica . . . 12

3.2.4 Sistema y Main . . . 12

3.3 Diagramas UML Actualizados . . . 13

3.4 Casos de Prueba y Metodología de Validación . . . 14

4 RESULTADOS Y ANÁLISIS DE EJECUCIÓN 16

4.1 Resultados . . . 16

5 CONCLUSIONES 17

6 REFERENCIAS 19

1. INTRODUCCIÓN

1.1. Planteamiento del Problema

Durante la primera etapa del Simulador de Sistema Hospitalario construimos una arquitectura preliminar conformada por cinco clases concretas (Sistema, Medico, Enfermero, Paciente y Main) alojadas dentro de un espacio de nombres plano. Aquel modelo inicial nos permitió registrar personal médico y de enfermería, administrar el ciclo de vida clínico del paciente a través de una máquina de estados finita (PENDIENTE, EN CONSULTA y EN TRATAMIENTO) y restringir la capacidad operativa mediante arreglos estáticos de 100 elementos para el hospital, 10 pacientes como máximo por cada médico y 3 pacientes por cada enfermero. No obstante, tratar a todos los facultativos, enfermeros y pacientes como instancias homogéneas de clases concretas únicas limitó la capacidad del sistema para reflejar la estructura jerárquica y las reglas de canalización de un hospital real.

En la operación clínica cotidiana, el personal y los usuarios poseen atribuciones, credenciales y protocolos de atención diferenciados según su grado de especialización. Un médico general (MedicoGeneral) atiende padecimientos primarios dentro de un consultorio físico numerado y está facultado para expedir certificados de salud a pacientes generales, mientras que un médico especialista (MedicoEspecialista) requiere contar con una cédula de especialidad adicional, el respaldo de un hospital certificador y la facultad de emitir dictámenes especializados para pacientes complejos. De manera análoga, el personal de enfermería se divide entre enfermeros especialistas (EnfermeroEspecialista), asignados a áreas de procedimientos específicas con autorización para suministrar medicamentos generales o controlados, y jefes de enfermería (JefeEnfermeria), quienes además de brindar tratamiento bajo el límite reglamentario de tres pacientes ejecutan labores de supervisión sobre la plantilla de enfermeros y coordinación de guardias departamentales. Asimismo, los pacientes ingresan bajo dos regímenes distintos: pacientes generales (PacienteGeneral) referidos desde una clínica de procedencia para consulta primaria, y pacientes de especialidad (PacienteEspecialidad) que presentan un diagnóstico clínico previo y un nivel numérico de prioridad de atención.

Resolver esta diversidad dentro del diseño plano de la Etapa 1 obligaría a saturar las clases originales con atributos condicionales nulos y ramificaciones lógicas rígidas que degradarían la cohesión del software y expondrían el estado interno a inconsistencias. Por ello, en esta segunda etapa abordamos la problemática de refactorizar el simulador hacia una arquitectura orientada a objetos avanzada, sustentada en el encapsulamiento con atributos privados y métodos de acceso controlado, la inmutabilidad de identificadores críticos, la partición modular en paquetes de Java, el diseño de clases base abstractas, la implementación de una interfaz distribuida en formato .jar y el uso de polimorfismo estático y dinámico.

1.2. Motivación

La decisión arquitectónica de incorporar clases abstractas e interfaces responde a la necesidad de desacoplar el núcleo administrativo del hospital respecto de los comportamientos clínicos particulares de cada subtipo de entidad. Al transformar las clases Paciente, Medico y Enfermero

en superclases abstractas no instanciables, impedimos a nivel de compilación la creación de objetos genéricos ambiguos en la memoria Heap y centralizamos en un único punto por jerarquía los atributos compartidos y los algoritmos invariantes heredados de la primera etapa, tales como la validación de cupos máximos en colecciones dinámicas, la actualización de los tres estados clínicos y el ordenamiento alfabético descendente por intercambio sobre copias de las listas. Con ello eliminamos la redundancia de código entre las subclases especializadas y garantizamos que cualquier extensión futura herede de forma automática las reglas de integridad del hospital.

De forma complementaria, la definición de la interfaz AtencionMedica y su empaquetamiento externo en un archivo binario .jar permiten establecer un contrato formal de comportamiento que unifica operativamente a dos jerarquías de herencia distintas (Medico y Enfermero). Dado que ambas ramas de profesionales administran listas de pacientes, aplican tratamientos y describen su modalidad de atención, la interfaz provee un supertipo polimórfico común que faculta al controlador Sistema para agrupar y auditar a todo el personal clínico dentro de una lista heterogénea `ArrayList<AtencionMedica>`. Esta separación entre el contrato público y los detalles de implementación garantiza la escalabilidad y mantenibilidad modular del simulador ante la eventual incorporación de nuevas categorías médicas o quirúrgicas.

1.3. Objetivos

1.3.1. Objetivo General

Continuar con el desarrollo del simulador de sistema hospitalario mediante la aplicación de los conceptos de encapsulamiento, paquetes, herencia y polimorfismo, incorporando clases abstractas e interfaces para representar diferentes tipos de personal médico y pacientes, conservando sus funcionalidades principales y los cupos individuales de 10 pacientes por médico y 3 por enfermero, ahora gestionados con listas dinámicas.

1.3.2. Objetivos Específicos

- Estructurar la organización física y lógica del código fuente mediante paquetes de Java (hospital.sistema, hospital.personal, hospital.pacientes y hospital.interfaces) bajo el árbol de directorios src, lib, doc y test, empaquetando la interfaz AtencionMedica en una biblioteca .jar independiente e integrando el uso explícito de las directivas package, import y nombres completamente cualificados (Fully Qualified Names).
- Implementar el principio de encapsulamiento avanzado en todas las clases del sistema utilizando atributos private y métodos de acceso controlado, incluido un modificador protected cuando corresponde, asegurando la inmutabilidad post-inicialización de identificadores oficiales (como la cédula profesional, la cédula de especialidad y el nombre del paciente) e incorporando filtros de validación en constructores y métodos modificadores (setters).
- Diseñar las tres jerarquías de herencia encabezadas por las clases abstractas Paciente, Medico y Enfermero, delegando la inicialización de estado común mediante la referencia super y especializando atributos y comportamientos propios en las subclases concretas Paciente-

General, PacienteEspecialidad, MedicoGeneral, MedicoEspecialista, EnfermeroEspecialista y JefeEnfermeria.

- Validar el funcionamiento del polimorfismo estático y dinámico almacenando objetos de diversas subclases en colecciones `ArrayList` referenciadas por su clase base abstracta o por la interfaz AtencionMedica, comprobando en tiempo de ejecución el enlace tardío de los métodos sobrescritos (darConsulta, darTratamiento, solicitarConsulta) y el cumplimiento estricto de la asignación por especialidad y límites de cupo.

2. MARCO TEÓRICO

2.1. Encapsulamiento y Protección de Estado

El encapsulamiento en la Programación Orientada a Objetos constituye el mecanismo arquitectónico que agrupa los datos que definen el estado interno de una entidad (sus atributos) junto con las operaciones que manipulan dicho estado (sus métodos), imponiendo al mismo tiempo una barrera de ocultamiento de información (information hiding) frente a componentes externos [1], [2]. En términos formales de ingeniería de software, si el estado válido de un objeto en un instante dado debe satisfacer un predicado invariante de clase, el ocultamiento de variables garantiza que ningún agente externo pueda modificar arbitrariamente las posiciones de memoria del objeto sin atravesar un punto único de contacto controlado por métodos de lectura y modificación [1], [5].

En el lenguaje Java, el control del alcance de los miembros de una clase dentro de una jerarquía de herencia y paquetes se instrumenta mediante especificadores de acceso evaluados por el compilador y verificados en tiempo de ejecución por la JVM [3]:

- **Visibilidad Privada (private):** Confiere el nivel más restrictivo de protección léxica [1], [3]. Los atributos declarados como private solo pueden ser leídos o modificados dentro del cuerpo de la misma clase que los declara; ni las clases externas ni las subclases derivadas tienen acceso directo a ellos, obligando a interactuar mediante métodos públicos o protegidos [1], [3].
- **Visibilidad Protegida (protected):** Establece un nivel de acceso intermedio diseñado para facilitar la extensión controlada en jerarquías de herencia [3], [4]. Un atributo o método declarado con protected permanece oculto e inaccesible para las clases clientes situadas en otros paquetes, pero puede ser accedido directamente por las subclases derivadas (incluso si se encuentran en paquetes distintos) y por las clases del mismo paquete [3]. En nuestra arquitectura hospitalaria, las clases abstractas Medico, Enfermero y Paciente mantienen privados sus atributos internos, incluidas las listas de pacientes, las especialidades y las referencias clínicas. Las subclases utilizan métodos de acceso; en Medico, setPacienteEnConsulta(Paciente) tiene visibilidad protected para permitir la actualización controlada de la consulta activa [3], [5].

El diseño riguroso del encapsulamiento exige además distinguir entre atributos mutables y atributos inmutables post-construcción [2], [5]. En un sistema hospitalario, credenciales oficiales como la cédula profesional de un médico o enfermero (cedula), la cédula de especialidad médica (cedulaEspecialidad) o el nombre titular de un paciente (nombre) constituyen identificadores permanentes que actúan como llaves primarias de indexación y búsqueda en los mapas de búsqueda

del controlador Sistema. Si una clase proporcionara un método modificador público como setCedula(), cualquier módulo externo podría alterar la identidad de un facultativo después de haber sido validado y registrado en el hospital, provocando duplicidades ocultas, pérdida de trazabilidad clínica y corrupción en las búsquedas por clave. Por esta razón, dichos atributos se declaran private, se validan y asignan por única vez dentro del constructor y carecen deliberadamente de método setter, exponiendo únicamente su método lector (getter) [2], [5].

Para aquellos atributos cuya semántica operativa sí requiere actualización a lo largo del tiempo, tales como el número de consultorio asignado a un médico general, el hospital certificador de un especialista, el área de procedimientos de un enfermero, el número de enfermeros bajo mando de un jefe de enfermería, la clínica de procedencia de un paciente general o el diagnóstico previo y nivel de prioridad de un paciente de especialidad, los métodos modificadores (setters) integran filtros de validación estrictos [1], [5]. Los setters rechazan cadenas nulas o vacías en diversos atributos, mientras que algunos constructores y setters sustituyen valores inválidos por valores predeterminados: por ejemplo, el consultorio se fija en 1 y el personal bajo mando en 0 cuando se proporciona un número fuera de rango. La prioridad clínica se limita a valores del 1 al 3, con 1 como valor predeterminado [1], [5].

2.2. Modularidad y Espacios de Nombres (Packages)

Cuando un sistema crece en complejidad y cantidad de clases, concentrar todos los archivos fuente dentro de un directorio único genera contaminación del espacio global de nombres, riesgo de colisión entre identificadores idénticos y acoplamiento desordenado entre componentes de distinta naturaleza [4], [6]. Java resuelve este problema mediante la organización en paquetes (packages), los cuales agrupan clases e interfaces funcionalmente relacionadas dentro de espacios de nombres jerárquicos que guardan correspondencia directa con el árbol de directorios del sistema operativo [4], [6].

Una estructura de proyecto modular separa físicamente los activos del software en cuatro directorios principales según su propósito en el ciclo de desarrollo [4]:

- **Directorio src/:** Contiene los subdirectorios correspondientes a los paquetes del código fuente (.java), separados por responsabilidades de dominio (sistema, personal, pacientes, interfaces).
- **Directorio lib/:** Aloja bibliotecas y componentes compilados empaquetados en archivos binarios .jar (Java Archive), permitiendo consumir contratos e interfaces como módulos reutilizables independientes [4], [7].
- **Directorio doc/:** Destinado a almacenar la documentación técnica de las clases, atributos y métodos generada mediante la utilería javadoc [4].
- **Directorio test/:** Reservado para alojar las clases y secuencias de prueba que validan la integración de los paquetes [4].

La interacción entre clases ubicadas en paquetes distintos se instrumenta mediante tres recursos sintácticos [4], [6]:

1. **Directiva package:** Declarada al inicio de cada archivo fuente (por ejemplo, package hospital.personal;), establece la pertenencia formal de la clase a ese espacio de nombres [4], [6].
2. **Directiva import:** Permite importar tipos públicos definidos en otros paquetes (como import hospital.pacientes.Paciente;) para referenciarlos mediante su nombre simple dentro del archivo actual [4], [6].
3. **Fully Qualified Names - FQN:** Consiste en invocar una clase o interfaz escribiendo su ruta jerárquica completa de paquetes separada por puntos directamente en la instrucción donde se requiere (por ejemplo, `hospital.interfaces.AtencionMedica`) [4], [6]. El empleo de FQN elimina cualquier ambigüedad cuando existen clases con el mismo nombre en paquetes diferentes y explicita en el propio punto de uso el origen arquitectónico del tipo referenciado [6].

Adicionalmente, la herramienta jar del JDK permite comprimir y empaquetar archivos de bytecode compilados (.class) junto con un archivo de manifiesto (MANIFEST.MF) dentro de una biblioteca .jar (Java Archive) [7]. Aislar la interfaz AtencionMedica en el archivo lib/AtencionMedica.jar y enlazarla durante la compilación y ejecución mediante el parámetro -cp (classpath) demuestra cómo externalizar contratos de software para su distribución modular sin exponer su código fuente [4], [7].

2.3. Jerarquías de Clases y Abstracción

La abstracción consiste en identificar las características esenciales y el comportamiento común de un conjunto de entidades para construir un modelo general que prescinda de los detalles particulares de cada variante [1], [8]. En la Programación Orientada a Objetos, esta organización conceptual se traduce en jerarquías de herencia, donde una superclase o clase base define la estructura general y un conjunto de subclases o clases derivadas heredan dichos atributos y métodos mediante la palabra reservada extends, estableciendo una relación taxonómica de tipo «es-un» (is-a) [8], [9].

Cuando una clase base representa un concepto general que no debe existir como objeto independiente en el sistema (dado que en el hospital ningún individuo es registrado como un «médico», «enfermero» o «paciente» abstracto, sino específicamente como médico general o especialista, enfermero especialista o jefe de enfermería, y paciente general o de especialidad), dicha superclase se declara como clase abstracta [8], [9]. Las clases abstractas Paciente, Medico y Enfermero cumplen tres funciones arquitectónicas en nuestro diseño [8], [9]:

- **Impedir la instanciación directa:** El compilador bloquea cualquier intento de crear instancias directas de la superclase abstracta, garantizando que en la memoria solo existan objetos de subclases concretas completamente definidas [8], [9].
- **Estandarizar y reutilizar estado y comportamiento:** Concentran los atributos comunes (nombre, cédula, especialidad, listas de pacientes y estado clínico, según la jerarquía) y los métodos cuya lógica es idéntica para las variantes correspondientes (registro en el sistema, adición de pacientes con validación de cupo y ordenamiento alfabético descendente), eliminando la duplicación de código [8].

- **Exigir especialización mediante métodos abstractos:** Declaran firmas de métodos con el modificador abstract sin cuerpo de implementación (tales como darConsulta en Medico, darTratamiento en Enfermero, o solicitarConsulta y getTipoPaciente en Paciente), obligando por contrato de compilación a que cada subclase concreta sobrescriba el método con su propio algoritmo especializado [8], [9].

En las clases derivadas, la reutilización del proceso de inicialización heredado se lleva a cabo mediante la palabra clave super [3], [8]. Al invocar super(...) como primera instrucción dentro del constructor de la subclase, se delega a la clase base abstracta la validación y asignación de los atributos generales y la creación de las listas internas cuando corresponde, antes de inicializar los atributos propios de la subclase [8]. Asimismo, el prefijo super. permite invocar métodos de la superclase desde métodos sobrescritos cuando se requiere extender un comportamiento conservando las validaciones base [3], [8].

2.4. Polimorfismo Dinámico y Late Binding

El polimorfismo es la propiedad mediante la cual una misma referencia o invocación de método puede adoptar múltiples formas y ejecutar comportamientos distintos dependiendo del tipo de las entidades involucradas [9], [10]. En nuestro sistema hospitalario coexisten dos mecanismos polimórficos complementarios [9], [10]:

- **Polimorfismo Estático (Sobrecarga / Overloading):** Consiste en definir dentro de una misma clase múltiples métodos o constructores con el mismo identificador pero con diferente lista de parámetros formales (en número o tipos de datos), como ocurre en la clase Sistema con el método sobrecargado asignarPaciente(Paciente, Medico) y asignarPaciente(Paciente, Enfermero), o en los dos constructores sobrecargados de la subclase MedicoGeneral [2], [10]. La selección de la versión a ejecutar es resuelta por el compilador en tiempo de compilación (early / static binding) según el tipo declarado de los argumentos [10].
- **Polimorfismo Dinámico (Sobrescritura / Overriding y Despacho por Interfaz):** Se produce cuando una subclase redefine un método heredado de su clase base abstracta o implementa un método definido en una interfaz conservando exactamente la misma firma (nombre, tipos de parámetros y retorno) [9], [10].

Gracias a la relación de subtipado (upcasting), una colección genérica declarada con referencias de una clase base abstracta (`ArrayList<Paciente>`, `ArrayList<Medico>`, `ArrayList<Enfermero>`) o de una interfaz común (`ArrayList<AtencionMedica>`) puede almacenar de manera heterogénea objetos instanciados de cualquiera de sus subclases concretas [9], [10]. Cuando el programa invoca un método polimórfico a través de una de estas referencias generales (por ejemplo, medico.darConsulta(paciente) o personal.getDescripcionAtencion()), la determinación de qué implementación ejecutar no depende del tipo estático de la variable de referencia, sino del tipo real del objeto en memoria mediante el mecanismo de enlace tardío (late / dynamic binding) en tiempo de ejecución [9], [10].

A nivel interno de la JVM, las llamadas sobre referencias de clase base se compilan como instrucciones invokevirtual y las llamadas sobre referencias de interfaz como invokeinterface [10]. En el instante exacto en que se ejecuta la instrucción en tiempo real, la JVM consulta la cabecera del objeto alojado en la memoria Heap, accede a su tabla de despacho de métodos virtuales (vtable) o tabla de interfaz (itable) y transfiere el flujo de ejecución hacia el método sobrescrito específico de la subclase concreta (MedicoGeneral, MedicoEspecialista, EnfermeroEspecialista, JefeEnfermeria, PacienteGeneral o PacienteEspecialidad) [9], [10].

3. DESARROLLO Y ARQUITECTURA DEL SISTEMA

3.1. Arquitectura de Paquetes y Distribución Modular

Para cumplir rigurosamente con la organización modular planteada en las especificaciones de la Etapa 2, estructuramos el proyecto bajo la jerarquía de directorios poo/g7/e0/py2/, con la separación prevista entre el código fuente (src/), las bibliotecas compiladas (lib/), la documentación (doc/) y las pruebas (test/). Dentro de src/hospital/, distribuimos las doce unidades de compilación en cuatro paquetes especializados según su responsabilidad arquitectónica:

- **Paquete hospital.interfaces (poo/g7/e0/py2/src/hospital/interfaces/):** Aloja la interfaz AtencionMedica.java. Su cabecera declara package hospital.interfaces; e importa hospital.pacientes.Paciente para definir el contrato asistencial estandarizado. Compilamos este componente y lo empaquetamos en el archivo binario poo/g7/e0/py2/lib/AtencionMedica.jar, distribuyendo el contrato de atención médica como una biblioteca independiente reutilizable.
- **Paquete hospital.pacientes (poo/g7/e0/py2/src/hospital/pacientes/):** Contiene la clase base abstracta Paciente.java y sus dos subclases especializadas PacienteGeneral.java y PacienteEspecialidad.java. Declara la directiva package hospital.pacientes; e incorpora sentencias import hacia hospital.personal.Medico, hospital.personal.Enfermero y hospital.sistema.Sistema para gestionar las referencias de asignación clínica y el registro en el hospital.
- **Paquete hospital.personal (poo/g7/e0/py2/src/hospital/personal/):** Agrupa las seis clases del personal de salud: la clase abstracta Medico.java con sus subclases MedicoGeneral.java y MedicoEspecialista.java, y la clase abstracta Enfermero.java con sus subclases EnfermeroEspecialista.java y JefeEnfermeria.java. Declara package hospital.personal;, importa las clases del paquete hospital.pacientes.* y implementa la interfaz AtencionMedica mediante import, mientras que su Nombre Completamente Cualificado (Fully Qualified Name - FQN) es `hospital.interfaces.AtencionMedica`.
- **Paquete hospital.sistema (poo/g7/e0/py2/src/hospital/sistema/):** Contiene el controlador central Sistema.java y la interfaz interactiva de consola Main.java bajo la directiva package hospital.sistema;. Utiliza directivas import para acceder a todas las clases de hospital.personal.* y hospital.pacientes.*, y emplea de forma explícita el FQN `hospital.interfaces.AtencionMedica` al construir y recorrer la lista polimórfica del catálogo de atención médica.

3.2. Descripción Algorítmica de la Jerarquía y Clases

A continuación, explicamos la arquitectura interna, atributos, justificaciones de encapsulamiento e inmutabilidad, constructores y algoritmos de todas las clases e interfaces del sistema.

3.2.1. Clases Abstractas Base (Paciente, Medico, Enfermero)

- **Clase Abstracta Paciente (hospital.pacientes):** Actúa como la plantilla base no instanciable para todos los usuarios del hospital. Declara el atributo nombre (String) con visibilidad private y carácter estrictamente inmutable ({readOnly}), inicializándolo en el constructor sin proveer un método setNombre() debido a que el nombre completo constituye la clave de identificación y búsqueda del expediente dentro del controlador Sistema. Los atributos especialidadAtencion (String), estado (String), medicoAsignado (Medico), enfermeroAsignado (Enfermero) y sistema (Sistema) se declaran con visibilidad private y se exponen de forma controlada mediante getters y setters; las subclases los utilizan a través de dichos métodos. Define asimismo las tres constantes públicas, estáticas e inmutables que controlan la máquina de estados: ESTADO_PENDIENTE = "PENDIENTE", ESTADO_EN_CONSULTA = "EN CONSULTA" y ESTADO_EN_TRATAMIENTO = "EN TRATAMIENTO". Su constructor recibe el nombre y la especialidad requerida, asigna valores predeterminados cuando el nombre o la especialidad son nulos o vacíos, establece el estado inicial en "PENDIENTE" y mantiene inicialmente en null las referencias de médico, enfermero y sistema. Implementa de forma concreta los métodos registroEnSistema(Sistema), los evaluadores booleanos de estado (estaPendiente(), estaEnConsulta(), estaEnTratamiento() y verTratamiento()) y declara como métodos abstractos obligatorios solicitarConsulta(Medico): boolean y getTipoPaciente(): String.
- **Clase Abstracta Medico (hospital.personal):** Define la superclase abstracta del cuerpo médico e implementa la interfaz `hospital.interfaces.AtencionMedica`. Declara con visibilidad private los atributos nombre (String) y cedula (String). Por justificación de diseño e integridad referencial, el atributo cedula es inmutable ({readOnly}) y carece de método setCedula(), puesto que la cédula profesional es una credencial oficial única e intransferible con la que el médico es indexado en el hospital; bloquear su modificación evita que un médico ya registrado altere su identidad o genere colisiones de claves en memoria. Los atributos especialidad (String), la lista pacientes (`ArrayList<Paciente>`), la referencia pacienteEnConsulta (Paciente) y el vínculo sistema (Sistema) tienen visibilidad private. El número de pacientes se obtiene mediante pacientes.size(), sin contador almacenado; las subclases acceden a la consulta activa mediante getters y el setter protegido setPacienteEnConsulta(Paciente). Conserva la constante estática CAPACIDAD_MAXIMA_PACIENTES = 10. Su constructor valida e inicializa los datos base, inicializa una lista ArrayList vacía y asigna null a pacienteEnConsulta. El método concreto agregarPaciente(Paciente) verifica que el paciente no sea nulo, que pacientes.size() sea menor a 10, que no esté duplicado en la lista del médico y que exista compatibilidad clínica de especialidad, añadiendo la referencia a la lista. El método darTratamiento(Paciente) verifica que el paciente esté asignado al médico, actualiza su estado a "EN TRATAMIENTO" y, si era el paciente que ocupaba pacienteEnConsulta, libera la consulta

asignando null directamente a pacienteEnConsulta para permitir que otro paciente pueda pasar a consulta. El método verListaPacientes() crea una copia auxiliar de la lista ArrayList y ejecuta el algoritmo de ordenamiento por intercambio (Bubble Sort) con compareToIgnoreCase() para devolver los nombres ordenados alfabéticamente de forma descendente (de la Z a la A) sin alterar el orden original en memoria. Finalmente, declara como abstracto darConsulta(Paciente): boolean; getDescripcionAtencion(): String se exige por la interfaz y lo implementan las subclases concretas.

- **Clase Abstracta Enfermero (hospital.personal):** Representa el molde abstracto del personal de enfermería e implementa igualmente la interfaz `hospital.interfaces.AtencionMedica`. Declara nombre y cedula con visibilidad private, manteniendo cedula estrictamente inmutable (sin método setCedula()) bajo la misma justificación de protección de credenciales oficiales. Los atributos especialidad (String), pacientes (`ArrayList<Paciente>`) y sistema (Sistema) poseen visibilidad private, respetando la constante estática CAPACIDAD_MAXIMA_PACIENTES = 3. El número de pacientes se calcula a partir de pacientes.size(). Su constructor inicializa los atributos base e inicializa una lista ArrayList vacía a la que se aplica el límite lógico de 3 pacientes. Implementa de manera concreta agregarPaciente(Paciente) (validando no nulidad, cupo menor a 3, ausencia de duplicados y coincidencia exacta de especialidad), getPacienteAsignado(Paciente) y el método verListaPacientes() con ordenamiento por intercambio alfabético descendente (Z-A) sobre una copia auxiliar de la lista. Declara abstracto darTratamiento(Paciente): boolean; getDescripcionAtencion(): String es una obligación de la interfaz implementada en cada subclase concreta.

3.2.2. Subclases Especializadas

- **Subclase PacienteGeneral (hospital.pacientes):** Extiende a la clase abstracta Paciente para modelar a los usuarios de consulta médica general. Incorpora el atributo propio privado clinicaProcedencia (String), el cual cuenta con método getter y método setter validado contra cadenas nulas o vacías, permitiendo actualizar la clínica familiar de referencia del usuario. Dispone de dos constructores: (nombre, clinicaProcedencia), que llama a super(nombre, "General"), y (nombre, especialidadAtencion, clinicaProcedencia), que llama a super(nombre, especialidadAtencion). Implementa el método propio obtenerFichaGeneral(): String para reportar la clínica de origen junto con los datos del expediente, sobrescribe getTipoPaciente() retornando la clasificación general y sobrescribe solicitarConsulta(Medico) validando que el paciente se encuentre en estado "PENDIENTE", que esté asignado al médico general y delegando la apertura de la consulta mediante la invocación polimórfica medico.darConsulta(this).
- **Subclase PacienteEspecialidad (hospital.pacientes):** Extiende a Paciente para representar a los usuarios que requieren atención de una especialidad médica específica. Incorpora dos atributos propios con visibilidad private: diagnosticoPrevio (String) y nivelPrioridad (int), ambos provistos de métodos getters y setters con validación de cadenas no vacías y prioridades comprendidas entre 1 y 3 (con valor 1 por defecto si se proporciona otra cantidad), dado que el cuadro diagnóstico y el nivel de urgencia pueden modificarse durante el seguimiento médico. Su constructor recibe cuatro parámetros e invoca a super(nombre, especialidadA-

tencion). Implementa los métodos propios obtenerResumenClinico(): String y actualizarDiagnostico(String), sobrescribe getTipoPaciente() y redefine solicitarConsulta(Medico) aplicando la restricción de negocio de la Etapa 2: verifica que el facultativo sea obligatoriamente una instancia de MedicoEspecialista y que su especialidad coincida exactamente con especialidadAtencion, rechazando cualquier intento de consulta con médicos generales o especialistas de un área distinta.

- **Subclase MedicoGeneral (hospital.personal):** Extiende a la clase abstracta Medico. Añade el atributo propio privado numeroConsultorio (int), dotado de getNumeroConsultorio() y setNumeroConsultorio(int) con validación de número entero estrictamente mayor a cero, ya que la administración hospitalaria puede reubicar el consultorio asignado al médico. Aplica polimorfismo estático mediante la sobrecarga de dos constructores: un primer constructor de tres parámetros (nombre, cedula, numeroConsultorio) que invoca a super(nombre, cedula, "General"), y un segundo constructor de cuatro parámetros (nombre, cedula, especialidad, numeroConsultorio) que invoca a super(nombre, cedula, especialidad). Incorpora el comportamiento propio expedirCertificadoSalud(PacienteGeneral): boolean, el cual verifica que la referencia al paciente general no sea nula antes de imprimir la expedición del certificado; este método no comprueba que el paciente pertenezca a su lista de asignación. Sobrescribe darConsulta(Paciente) validando que el paciente sea de atención general, que esté previamente asignado dentro de su lista de hasta 10 pacientes, que su estado sea "PENDIENTE" y que el consultorio no esté ocupado (pacienteEnConsulta == null); si ya existe un paciente en consulta activa, imprime el mensaje "Aviso: El medico general ya tiene un paciente en consulta." y retorna false, mientras que si está disponible invoca setPacienteEnConsulta(paciente) y actualiza el estado del paciente a "EN CONSULTA". Sobrescribe además getDescripcionAtencion() retornando "Atencion Medica General - Consultorio " concatenado con numeroConsultorio.
- **Subclase MedicoEspecialista (hospital.personal):** Extiende a Medico para modelar a los facultativos de especialidad. Incorpora dos atributos privados propios: cedulaEspecialidad (String) y hospitalCertificador (String). Siguiendo el criterio de protección de credenciales oficiales, cedulaEspecialidad es estrictamente inmutable y solo posee el método lector getCedulaEspecialidad() (sin setter), mientras que hospitalCertificador cuenta con getter y setter validado. Su constructor de cinco parámetros invoca a super(nombre, cedula, especialidad) e inicializa ambos atributos de certificación. Implementa el comportamiento exclusivo emitirDictamenEspecializado(PacienteEspecialidad, String): String, que registra el dictamen médico sobre el paciente de especialidad y retorna el informe con su cédula de especialidad; además, actualiza el diagnóstico del paciente, pero el texto devuelto no incluye el hospital certificador. Sobrescribe darConsulta(Paciente) exigiendo que el usuario sea una instancia de PacienteEspecialidad de su misma rama médica, que esté en estado "PENDIENTE" dentro de su lista de hasta 10 pacientes y que pacienteEnConsulta sea null, además de sobrescribir darTratamiento(Paciente) y getDescripcionAtencion().
- **Subclase EnfermeroEspecialista (hospital.personal):** Extiende a la clase abstracta Enfermero. Declara el atributo propio privado areaProcedimientos (String) con sus métodos getAreaProcedimientos() y setAreaProcedimientos(String) validados, e invoca a super(nombre, cedula, especialidad) en su constructor. Implementa dos comportamientos propios: admi-

nistrarMedicamentoControlado(Paciente, String, String): boolean y suministrarMedicamento(Paciente, String, String): boolean, los cuales verifican que el paciente esté asignado dentro de su cupo de 3 pacientes, y que los datos de medicamento y dosis no sean nulos ni vacíos (la especialidad se comprueba previamente al asignar al paciente). Sobrescribe darTratamiento(Paciente) actualizando el estado del paciente asignado a "EN TRATAMIENTO" dentro de su área de procedimientos, y sobrescribe getDescripcionAtencion().

- **Subclase JefeEnfermeria (hospital.personal):** Extiende a Enfermero para representar el nivel jerárquico de supervisión y coordinación de enfermería. Añade los atributos privados departamentoCoordinado (String) y enfermerosBajoMando (int), provistos de métodos getters y setters con validación de cadenas no vacías y cantidades enteras mayores o iguales a cero. Su constructor invoca a super(nombre, cedula, especialidad) e inicializa ambos atributos. Cumple con el requisito de supervisión mediante dos métodos propios: supervisarPersonal(`ArrayList<Enfermero>`): String, que recibe la lista de enfermeros del hospital para auditar su carga de pacientes y especialidad; también conserva una sobrecarga supervisarPersonal(Enfermero[], int): String, que convierte un arreglo a lista; y coordinarGuardia(Enfermero, String): boolean, que comprueba que el enfermero y el turno no sean nulos o vacíos y devuelve un booleano, sin almacenar permanentemente una guardia. Sobrescribe darTratamiento(Paciente) conservando el límite de 3 pacientes asignados y getDescripcionAtencion().

3.2.3. Interfaz AtencionMedica

Definida en el paquete hospital.interfaces y empaquetada en lib/AtencionMedica.jar, establece el contrato de comportamiento estandarizado que implementan las jerarquías de Medico y Enfermero a través de tres firmas abstractas públicas: darTratamiento(Paciente paciente): boolean, verListaPacientes(): String y getDescripcionAtencion(): String. Esta interfaz desacopla al controlador Sistema de las clases concretas del personal asistencial, permitiendo tratar de manera uniforme a médicos generales, médicos especialistas, enfermeros especialistas y jefes de enfermería.

3.2.4. Sistema y Main

- **Clase Controladora Sistema (hospital.sistema):** Administra el estado global del hospital mediante el atributo privado nombreHospital (String), tres colecciones dinámicas privadas: medicos (`ArrayList<Medico>`), enfermeros (`ArrayList<Enfermero>`) y pacientes (`ArrayList<Paciente>`), más tres mapas de índice (`HashMap<String, Medico>`, `HashMap<String, Enfermero>` y `HashMap<String, Paciente>`). Los recuentos se obtienen mediante size() en vez de mantener contadores independientes. Los métodos registroMedico(Medico), registroEnfermero(Enfermero) y registroPaciente(Paciente) verifican que la referencia recibida no sea nula, que no exista duplicidad de cédula o nombre mediante las claves normalizadas con trim().toUpperCase() en los mapas HashMap, almacenando en una misma lista cualquier subclase de su respectiva jerarquía; el código actual no impone un máximo global de 100 registros. Conserva la sobrecarga estática de asignarPaciente(Paciente, Medico) y asignarPaciente(Paciente, Enfermero), integrando el filtro de negocio que asegura

que los objetos PacienteGeneral sean atendidos por instancias de MedicoGeneral y que los objetos PacienteEspecialidad se asignen a médicos especialistas y que las asignaciones de enfermería respeten la especialidad; EnfermeroEspecialista rechaza pacientes generales, mientras que JefeEnfermeria puede recibir pacientes compatibles con su especialidad. Incorpora además el método getPersonalAtencion(): `ArrayList<AtencionMedica>`, el cual construye y retorna una lista polimórfica del tipo de la interfaz `hospital.interfaces.AtencionMedica` que consolida a todos los médicos y enfermeros registrados en el hospital.

- **Clase Ejecutable Main (hospital.sistema):** Actúa como la vista interactiva en consola, declarando una referencia estática a Sistema y gestionando la captura segura de datos mediante los métodos auxiliares privados leerTexto(Scanner, String) y leerEntero(Scanner, String). Despliega mediante un bucle do-while un menú estructurado de 14 opciones más la opción 0 de salida: 1) Registrar médico (MedicoGeneral o MedicoEspecialista), 2) Registrar enfermero (EnfermeroEspecialista o JefeEnfermeria), 3) Registrar paciente (PacienteGeneral o PacienteEspecialidad), 4) Asignar paciente a médico, 5) Asignar paciente a enfermero, 6) Solicitar consulta médica, 7) Dar tratamiento por médico, 8) Dar tratamiento por enfermero, 9) Ver lista de pacientes de un médico (ordenada de la Z a la A), 10) Ver lista de pacientes de un enfermero (ordenada de la Z a la A), 11) Ver estado clínico de un paciente, 12) Supervisión de enfermería (JefeEnfermeria), 13) Expedir certificado de salud (MedicoGeneral), y 14) Resumen general y catálogo de atención médica (`ArrayList<AtencionMedica>`).

3.3. Diagramas UML Actualizados

La estructura estática y las relaciones jerárquicas del sistema evolucionado se documentan en el diagrama de clases UML de la Etapa 2, el cual integra las doce clases e interfaces distribuidas en sus respectivos paquetes, aunque sus anotaciones de arreglos y multiplicidades corresponden a la representación anterior del sistema.

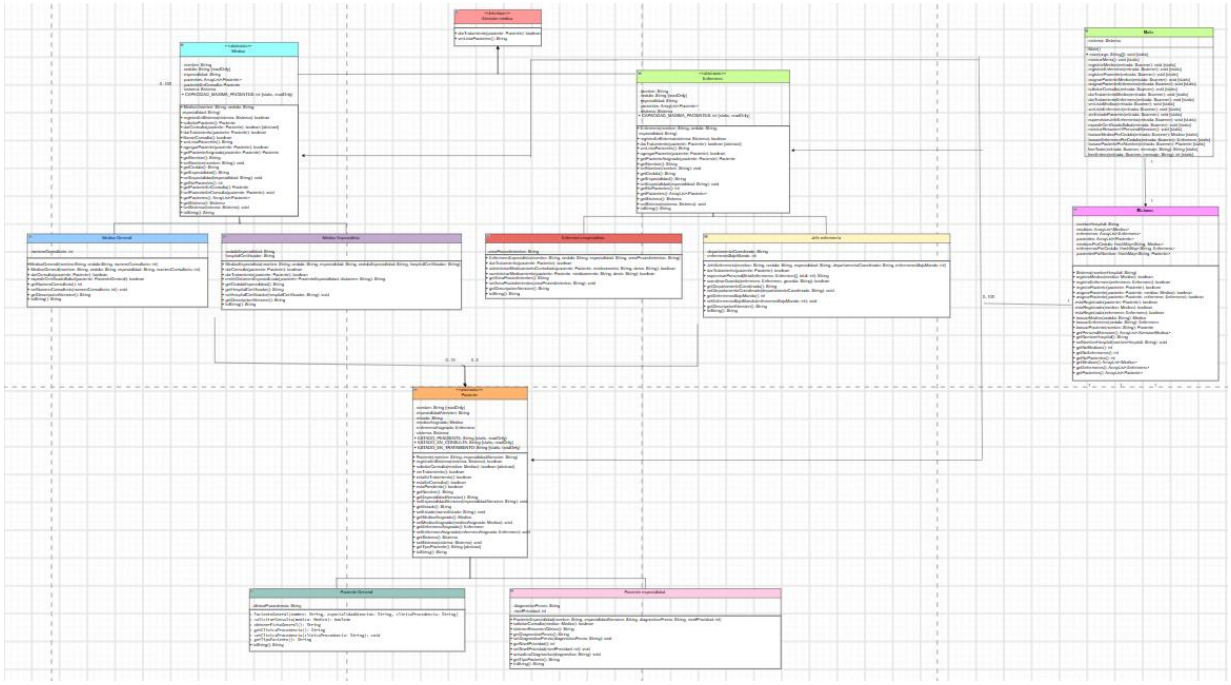


Figura 1. Diagrama de clases UML actualizado del Simulador de Sistema Hospitalario (Etapa 2), donde se especifican los controladores Main y Sistema, la interfaz «Interface» Atención médica, las superclases abstractas («abstracto» Médico, «abstracto» Enfermero, «abstracto» Paciente), las seis subclases especializadas (MedicoGeneral, MedicoEspecialista, EnfermeroEspecialista, JefeEnfermeria, PacienteGeneral, PacienteEspecialidad), el encapsulamiento mediante modificadores de visibilidad (-, #, +), los atributos inmutables ({readOnly}) y las multiplicidades históricas (0..100, 0..10 y 0..3) del diagrama; el código actual usa listas dinámicas, sin límite global de 100.

Enlace al diagrama original: https://app.diagrams.net/#G1AORzx2K712YzUOEzN9llkGmixtHhkFnq#%7B%22pageId%22%3A%22 qpc1kE9T77E4vkm_M2_r %22 %7D

3.4. Casos de Prueba y Metodología de Validación

Para validar integralmente el cumplimiento de los requerimientos funcionales y arquitectónicos de la Etapa 2, ejecutamos una metodología de pruebas sistemáticas en consola dividida en siete escenarios representativos que cubren tanto la operación nominal como las restricciones de borde del hospital:

- **Caso de Prueba 1 — Registro polimórfico de MedicoGeneral:** Iniciamos el simulador con el nombre "Hospital San Rafael", seleccionamos la opción 1 (Registrar médico), elegimos el subtipo 1 (Medico General) e ingresamos el nombre "Santos Mariana", la cédula "MED104" y el número de consultorio 204. El sistema invocó el constructor de MedicoGeneral, delegó a super("Santos Mariana", "MED104", "General") y almacenó la referencia por upcasting en la lista `ArrayList<Medico>` de Sistema, confirmando en pantalla: "Medico registrado exitosamente.".
- **Caso de Prueba 2 — Rechazo automático por duplicidad e inmutabilidad de cédula profesional:** Seleccionamos nuevamente la opción 1, elegimos el subtipo 1 (Medico

General) e intentamos registrar al médico "Torres Felipe" ingresando en minúsculas la cédula "med104" y el consultorio 208. El método registroMedico de Sistema consultó en HashMap una clave normalizada mediante trim().toUpperCase() contra la cédula inmutable "MED104" de la Dra. Santos Mariana, detectó la colisión e impidió el registro imprimiendo: "Aviso: Ya existe un medico registrado con la cedula: med104" y "No fue posible registrar al medico.".

- **Caso de Prueba 3 — Registro polimórfico de instancias de PacienteGeneral:** A través de la opción 3 (Registrar paciente) y el subtipo 1 (Paciente General), dimos de alta a dos usuarios: primero a "Zamora Julia" con clínica de procedencia "Clinica Norte", y segundo a "Castillo Pedro" con clínica de procedencia "Clinica Central". Ambas instancias se inicializaron en estado "PENDIENTE" y se almacenaron dentro de la lista polimórfica `ArrayList<Paciente>` del controlador con el mensaje "Paciente registrado exitosamente.".
- **Caso de Prueba 4 — Asignación compatible y ordenamiento alfabético descendente (Z-A):** Mediante la opción 4 asignamos tanto a "Zamora Julia" como a "Castillo Pedro" a la cartera de la médica "MED104" ("Santos Mariana"), verificando la compatibilidad entre PacienteGeneral y MedicoGeneral y el límite menor a 10 pacientes. Posteriormente ejecutamos la opción 9 (Ver lista de pacientes de un médico) para la cédula "MED104", comprobando que el método verListaPacientes() ordenó una copia de la lista de forma alfabética descendente (Z-A) imprimiendo primero "Zamora Julia - General" y segundo "Castillo Pedro - General".
- **Caso de Prueba 5 — Solicitud de consulta polimórfica y candado de exclusividad de consultorio:** En la opción 6 (Solicitar consulta médica), solicitamos consulta para "Zamora Julia" con la médica "MED104", ejecutándose por enlace tardío PacienteGeneral.solicitarConsulta y MedicoGeneral.darConsulta, lo que cambió el estado de la paciente a "EN CONSULTA". Inmediatamente después, ejecutamos la opción 6 intentando iniciar consulta para "Castillo Pedro" con la misma médica "MED104"; el sistema verificó que pacienteEnConsulta no era nulo y bloqueó la operación mostrando: "Aviso: El medico general ya tiene un paciente en consulta." y "No fue posible iniciar la consulta medica.".
- **Caso de Prueba 6 — Validación de rechazos por incompatibilidad de especialidad y límites de capacidad:** Sometimos a prueba las reglas cruzadas intentando asignar un PacienteEspecialidad (por ejemplo, de "Cardiologia") a un MedicoGeneral o a un MedicoEspecialista de diferente área ("Oncologia"), así como intentos de superar el límite de 10 pacientes en médicos o 3 pacientes en enfermeros (EnfermeroEspecialista y JefeEnfermeria). En todos los casos los filtros de asignarPaciente y agregarPaciente rechazaron las operaciones inválidas conservando intacto el estado de las listas.
- **Caso de Prueba 7 — Catálogo polimórfico de atención médica y cierre del sistema:** Ejecutamos la opción 14 (Resumen general y catálogo de atención médica), donde el sistema reportó las métricas globales del "Hospital San Rafael" (Medicos registrados: 1, Enfermeros registrados: 0, Pacientes registrados: 2) y recorrió la lista polimórfica `ArrayList<AtencionMedica>` invocando por enlace tardío getDescripcionAtencion(), imprimiendo "[01] Atencion Medica General - Consultorio 204", antes de finalizar limpiamente con la opción 0 ("Sistema hospitalario cerrado correctamente.").

4. RESULTADOS Y ANÁLISIS DE EJECUCIÓN

4.1. Resultados

A continuación, presentamos la ejecución en terminal del simulador para el Hospital San Rafael.

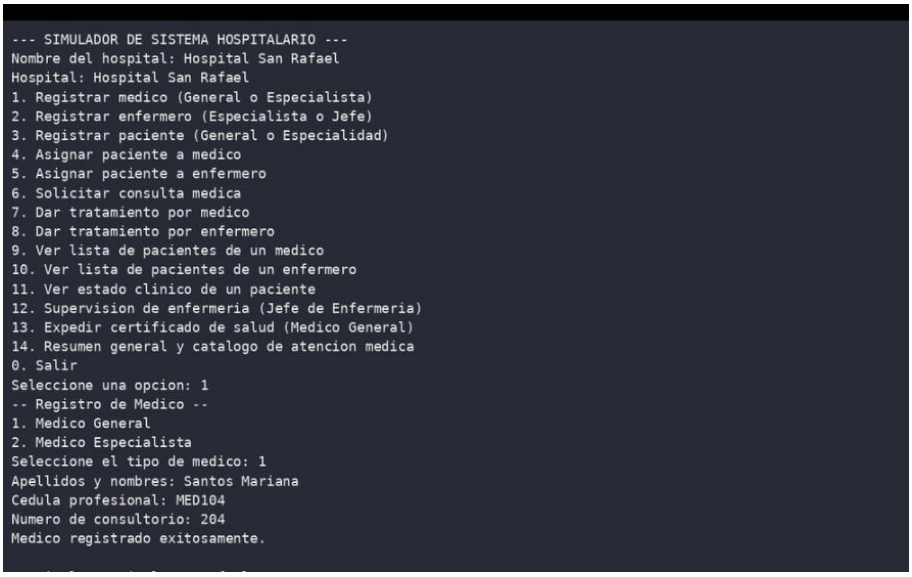


Figura 2. Inicialización del simulador para el "Hospital San Rafael", despliegue del menú interactivo de 14 opciones y registro polimórfico exitoso de un objeto de la subclase MedicoGeneral ("Santos Mariana", cédula "MED104", consultorio 204) en la lista de médicos del sistema.

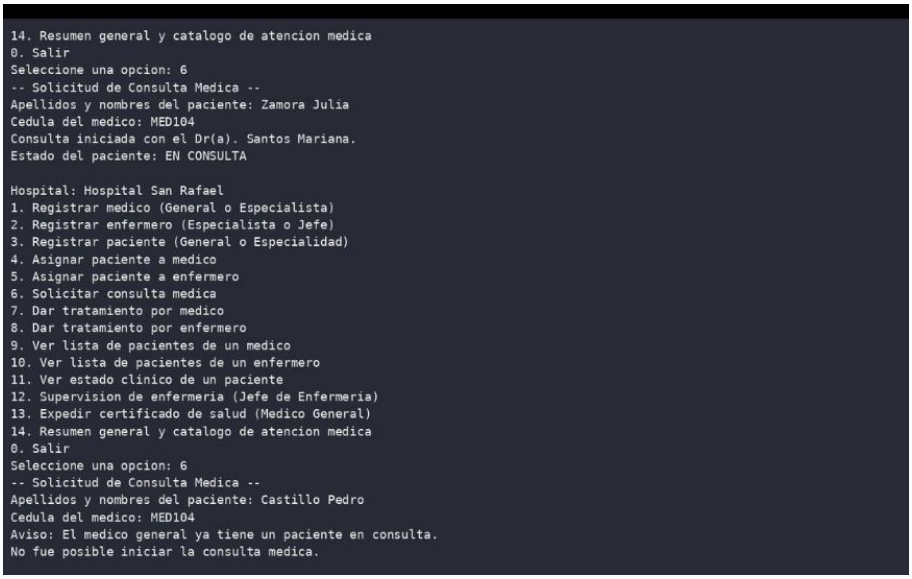


Figura 3. Asignación de ambos pacientes generales a la Dra. "Santos Mariana" (opción 4), visualización de la lista de pacientes ordenada alfabéticamente de forma descendente de la Z a la A (opción 9), inicio polimórfico de la consulta de "Zamora Julia" pasando a estado "EN CONSULTA" (opción 6) y rechazo automático de la solicitud simultánea de "Castillo Pedro" por encontrarse ocupado el consultorio.

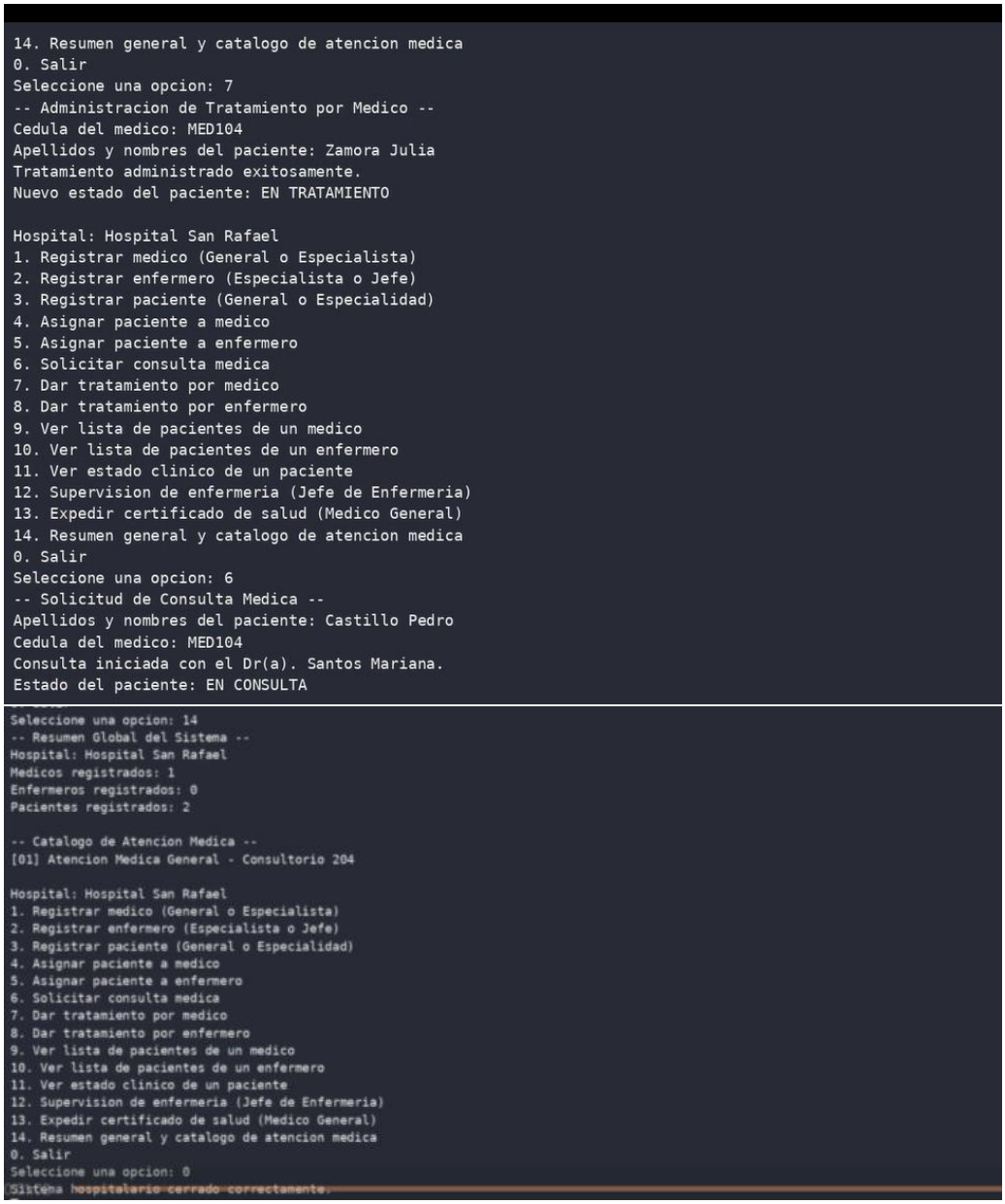


Figura 4. Transición de "Zamora Julia" al estado "EN TRATAMIENTO" mediante la opción 7 con liberación automática del consultorio de la Dra. "Santos Mariana", ingreso inmediato de "Castillo Pedro" al estado "EN CONSULTA" en la opción 6, e impresión en la opción 14 del resumen global del sistema y del catálogo polimórfico generado a través de la interfaz AtencionMedica.

5. CONCLUSIONES

La culminación de la segunda etapa del Simulador de Sistema Hospitalario nos permitió demostrar cómo la aplicación articulada del encapsulamiento, la modularidad por paquetes, la herencia mediante clases abstractas y el polimorfismo dinámico resuelve las limitaciones estructurales de un diseño plano y dota al software de solidez arquitectónica, seguridad interna y capacidad de evolución continua. A través de la instrumentación rigurosa de los niveles de visibilidad private y

un setter protected de uso interno, establecimos un blindaje efectivo sobre el estado interno de cada componente que impide manipulaciones externas arbitrarias, asegurando mediante decisiones justificadas de diseño que los identificadores críticos de carácter oficial (como la cédula profesional de médicos y enfermeros, la cédula de especialidad médica y el nombre titular del paciente) permanezcan estrictamente inmutables una vez inicializados por sus constructores, mientras que los atributos sujetos a cambios operativos o clínicos se actualicen únicamente bajo filtros de validación que rechazan o sustituyen por valores predeterminados los datos nulos, vacíos o fuera de rango. Paralelamente, la reestructuración física y lógica del proyecto en los paquetes hospital.sistema, hospital.personal, hospital.pacientes y hospital.interfaces, sumada a la compilación y empaquetamiento externo del contrato AtencionMedica dentro de una biblioteca binaria .jar y al uso preciso de las directivas package, import y nombres completamente cualificados, erradicó el acoplamiento monolítico inicial y garantizó una separación clara de responsabilidades entre la lógica de control, las entidades clínicas y las especificaciones de servicio. En el ámbito del modelado taxonómico, la transformación de las clases Paciente, Medico y Enfermero en superclases abstractas eliminó la posibilidad de instanciar entidades genéricas incompletas en la memoria y nos permitió centralizar los algoritmos fundamentales heredados de la primera etapa (tales como el respeto a los límites de diez pacientes por médico y tres por enfermero en listas dinámicas, la actualización de los estados clínicos y la exclusividad de consulta activa y el ordenamiento alfabético descendente por intercambio sobre copias seguras), delegando la construcción base mediante la referencia super y especializando en las seis subclases concretas los atributos y comportamientos propios de cada rol, desde la expedición de certificados de salud y dictámenes especializados hasta el suministro de medicamentos controlados y la supervisión departamental de enfermería. Finalmente, el empleo del polimorfismo dinámico y el enlace tardío sobre colecciones heterogéneas referenciadas por sus clases base abstractas y por la interfaz común AtencionMedica confirió al sistema la flexibilidad necesaria para almacenar múltiples variantes de objetos en estructuras unificadas y ejecutar en tiempo de ejecución el comportamiento exacto correspondiente al tipo real de cada instancia, consolidando una arquitectura hospitalaria modular, escalable y mantenible a largo plazo.

6. REFERENCIAS

[1] Universidad Nacional Autónoma de México (UNAM CUAED), "Abstracción y Encapsulamiento," Repositorio de la Coordinación de Universidad Abierta y Educación a Distancia. [En línea]. Disponible en: https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3076/mod_resource/content/1/UAPA-Abstraccion-Encapsulamiento/index.html

[2] Facultad de Ingeniería, Universidad Nacional Autónoma de México (FI UNAM), Manual de Prácticas del Laboratorio de Programación Orientada a Objetos (Lineamientos LCP02), Ciudad Universitaria, México: División de Ingeniería Eléctrica, Departamento de Ingeniería en Computación, UNAM, 2026. [En línea]. Disponible en: http://lcp02.fi-b.unam.mx/

[3] Oracle Corporation, "Controlling Access to Members of a Class," The Java Tutorials: Learning the Java Language — Classes and Objects. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html

[4] Oracle Corporation, "Creating and Using Packages," The Java Tutorials: Learning the Java Language — Packages. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/java/package/index.html

[5] J. López Blasco, "Introducción a POO en Java: Encapsulamiento," OpenWebinars, Nov. 10, 2023. [En línea]. Disponible en: https://openwebinars.net/blog/introduccion-a-poo-en-java-encapsulamiento/

[6] W3Schools, "Java Packages and API," W3Schools Online Web Tutorials. [En línea]. Disponible en: https://www.w3schools.com/java/java_packages.asp

[7] Oracle Corporation, "Packaging Programs in JAR Files," The Java Tutorials: Deployment — JAR Files. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/deployment/jar/index.html

[8] Oracle Corporation, "Abstract Methods and Classes," The Java Tutorials: Learning the Java Language — Interfaces and Inheritance. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html

[9] Universidad Nacional Autónoma de México (UNAM CUAED), "Herencia y Polimorfismo en la Programación Orientada a Objetos," Unidades de Apoyo para el Aprendizaje (UAPA), CUAED UNAM. [En línea]. Disponible en: https://uapa.cuaed.unam.mx/facultad_de_ingenieria

[10] Baeldung, "Polymorphism in Java," Baeldung Computer Science & Java Core Tutorials. [En línea]. Disponible en: https://www.baeldung.com/java-polymorphism